

2026-08-02. Written after three experiments in one evening, two of which were nulls and the third of which is the reason this document exists.

MAG chooses moves by scoring each candidate against a fitted vector. PORY judges positions. Neither looks forward. Everything measured tonight says the ceiling is **not** the feature set of either model, it is that **nothing in this project has ever considered a position it was not already standing in.**

The evidence is three measurements with the same shape and opposite verdicts:

experiment	what changed	accuracy moved
PORYGON2 retrained on a corrected bot	sampling→greedy, closed→open sheets, mega abilities fixed on 25% of Pokemon	+0.11
per-Pokemon HP distribution added	the weakest active body, not the mean	+0.03
one turn of foresight	score the next turn instead of this one	+4.91

Two years of feature work is implied by the first two rows. The third row is a day's work and is forty times larger.

`engine/lookahead_bound.py` . Train the k-NN value function on 98,776 self-play positions from the current bot; test on 70,446 aligned (turn t, turn t+1) pairs from 4,779 clean human games. Both columns use the same model, the same games, the same labels. The only difference is which turn's state the scorer is handed.

k=50	score the CURRENT turn	58.62%	Brier 0.2418	logloss 0.6862
k=50	score the NEXT turn (oracle)	63.55%	Brier 0.2198	logloss 0.6337
k=200	score the CURRENT turn	59.56%	Brier 0.2353	logloss 0.6635
k=200	score the NEXT turn (oracle)	64.46%	Brier 0.2150	logloss 0.6163

The oracle row is cheating and that is the point. It is handed the turn that actually happened, which no search can do — a search must *estimate* the futures and average over an opponent who chooses simultaneously. So +4.91 is a ceiling on what one step could ever be worth on top of this value function, not a forecast.

A null here would have been worth more than the search itself, because it would have killed the search for the price of an afternoon. It is not a null.

One-step lookahead over a base policy is *policy iteration truncated to one step* ([Bertsekas, Rollout Algorithms for Discrete Optimization](#)). The Policy Improvement Theorem guarantees the rollout policy is at least as good as the base policy, $v_{\text{rollout}}(s) \geq v_{\text{base}}(s)$, strictly better unless already optimal. Tesauro coined "rollout" for backgammon, where a simulator plays positions out and averages the results.

The precondition matters more than the theorem, and we do not meet it. Rollout inherits monotone improvement *because it starts from the true value of a policy*. Lookahead from an **arbitrary approximation carries no such guarantee** — improvement holds only under consistency conditions on the approximation. PORY is an arbitrary approximation: a k-NN fitted to outcomes, not the value function of any policy we actually run.

So there is no theorem here, only an empirical question — which is precisely why §2 was measured before anything was designed. Anyone quoting "rollout is guaranteed to improve the policy" as justification for this build is quoting a theorem whose hypothesis we fail.

This is the structural fact that decides the design. Both players commit without seeing the other, so "pick the action with the best successor value" is not defined — the value of my action depends on theirs. The correct object at each decision is a **payoff matrix**, and the correct answer is its **equilibrium**, not its maximum.

The MCTS literature for simultaneous-move games (Lanctot et al., *MCTS Variants for Simultaneous Move Games*; Bosansky et al., *Algorithms for Computing Strategies in Two-Player Simultaneous Move Games*) gives the options: Sequential UCT, **Decoupled UCT (DUCT)**, Exp3, and **Regret Matching**. DUCT has each player select independently from its own statistics and performs well empirically — Decoupled UCB1-Tuned won 62.3% in Tron — but **its asymptotic convergence to Nash is not guaranteed**. Regret matching does converge, and is the same principle as CFR minus the extensive-form tree.

We already have the regret-matching solver. `engine/slowking/nash.py:solve_rm` — Hart & Mas-Colell, dependency-free, converging to a few 1e-3 in a few thousand iterations — is used today for SLOWKING's team-preview stage game and is verified against Rock-Paper-Scissors by `tests/test-slowking.py`. Applying it at a mid-game turn is a new caller, not a new solver.

`docs/LITERATURE-v2.md` §2 already records it. **Metamon** (arXiv 2504.04395, RLC 2025) reaches top-10% of active human players with offline RL on 5M+ human trajectories plus 20M+ self-play, using a **black-box sequence model with NO explicit search** — and outperforms both an LLM agent and a strong heuristic search engine. **VGC-Bench** (arXiv 2506.10326) puts the doubles configuration space at $\sim 10^{139}$ and finds the central tension is exploitability versus generalisation across teams.

This is the strongest argument *against* the design below, and it is recorded here rather than buried: the current state of the art in this exact game got where it is without search. The honest position is that §2 measures a gap **in our value function**, and Metamon's route closes the same gap a different way. Both are live; this document argues only that the search is worth its cost, not that it is the only path.

At a decision point, each side chooses a **joint action** — both slots at once. Build the matrix

$$A[i][j] = V(\text{result of my joint action } i \text{ against their joint action } j)$$

and solve it. `solve_rm(A)` returns the equilibrium mixture for each side and the game value. Play from our mixture.

This is the same object SLOWKING already solves at team preview, one level down. That is not a coincidence to note in passing — it means the strategic and tactical layers share a solver, and a bug in it fails two tests instead of none.

piece	what it does	already built
candidate pruning	reduce the joint action space to top-K per side	<code>engine/fit_joint.js</code> already caps at top-6 per slot; <code>magnemite.js:_decidePair</code> builds and scores joint candidates today
the transition	apply one turn and produce the next board	<code>champions_sim</code> — the real engine, ADR-001, already pinned and verified
the leaf	score the resulting position	PORY / PORYGON2
the solver	equilibrium of the matrix	<code>engine/slowking/nash.py:solve_rm</code>

Nothing new is invented. The build is the wiring plus a cost bound.

A doubles side chooses, per slot, from roughly 4 moves x 2 targets plus up to 4 switches ~ 12 options, and there are two slots: **~144 joint actions per side**, so **~20,000 cells** per turn. Each cell is a simulated turn. That is not affordable at 10 games/sec.

With top-6 per slot, as `fit_joint` already uses: 36 joint actions per side, **1,296 cells**. With top-3: 9 per side, **81 cells** — affordable, and `fit_joint` measured that the human's chosen pair falls outside the top-6-per-slot window only **11.3%** of the time, which bounds what the pruning throws away.

MAG is the policy prior here, and that is its job in this architecture. This is the AlphaZero shape — a policy head to order and prune, a value head to score leaves — and it reframes tonight's undecided feature H2H: a better prior is worth less as a *player* than as a *pruner*, and it has never been measured as a pruner.

A Pokemon turn is not deterministic: damage rolls, accuracy, criticals, speed ties. One simulation of a cell is one sample of its value. Averaging n samples per cell multiplies cost by n . The honest first version takes **$n=1$** and reports the resulting variance rather than pretending it is a deterministic transition — with a measured check on how often the equilibrium mixture changes between two independent evaluations of the same board.

-
- **The bound is an upper bound.** A search recovers some fraction of +4.91, and the fraction is unknown. If it recovers a fifth, that is +1 accuracy point of position judgement — which may not survive contact with a win-rate H2H, exactly as tonight's feature work did not.
 - **The leaf is weak.** 63-64% accuracy, dominated by `foe_alive` at 2.6x the next feature, with four of seventeen features at exactly 0.00 importance. A search maximising a value function that mostly counts Pokemon is a slower way to count Pokemon.
 - **No policy-improvement guarantee** (§3.1).
 - **Metamon got further without any of this** (§3.3).
 - **Cost.** 81-1,296 simulated turns per decision against a live battle timer.
-

Each is a measurement, not an opinion, and each must pass before the next is built.

1. **G1 — the bound is real.** `engine/lookahead_bound.py` reports a mean oracle gain > 1.0 accuracy point.
PASSED: +4.91.
2. **G2 — the transition is affordable. RESOLVED 2026-08-03 — §8 supersedes everything in this item.** The `Infinite loop` was found and fixed, the failure rate is 0, and the cost figures below are wrong: they were measuring the bug. Left in place unedited because the reasoning was sound and only the diagnosis was not, and because §8 is only legible against it. `engine/lookahead_cost.js`.
`Battle.toJSON()/fromJSON()` round-trip a real mid-game state — turn 4 against turn 4, 139 log lines against 139, 31 KB — and forking plus stepping one turn costs **4.6 ms**. Checked rather than trusted: a fork that silently dropped a volatile or a boost would make every leaf wrong in the same direction, which is the shape of every expensive bug here.

matrix	cells	time	
unpruned (~144 joint actions/side)	20,736	95.9 s	hopeless
top-6 per slot (what <code>fit_joint</code> uses)	1,296	6.0 s	too slow
top-3 per slot	81	0.4 s	affordable

Those times are from one board and the cost is not stable. Across six runs the per-successful- fork cost ranged **3.5 to 32.2 ms** — a factor of nine, driven by the board rather than the machine. At the top of that range even top-3 costs 2.6 s of the 3 s budget, so the honest summary is: top-3 is affordable on a typical board and marginal on a bad one, top-6 is not affordable on any board measured, and the single-board figures above should not be quoted as the cost.

So §4.3 is wrong and this supersedes it: the search must prune to top-3 per slot, not top-6. That is a real cost, and it is unmeasured — `fit_joint` established that the human's chosen pair falls outside the top-6 window 11.3% of the time, and the equivalent figure for top-3 is not known. It should be measured

before G3, because it bounds how much of the +4.91 is reachable at all: a search cannot recover value from a branch it never enumerated. At one sample per cell; averaging n samples multiplies all of it by n (§4.4).

BUT A QUARTER TO A THIRD OF FORKS THROW `Infinite loop`. Measured across five runs: 40/40, 27/40, 30/40, 24/40, 25/40 succeeded, and cost per SUCCESSFUL fork ranged 3.5 to 15.0 ms. The first run was 40/40, which is the only reason this gate was briefly written up as passed — it was a lucky board, and the honest figure is a 25–40% failure rate under naive handling. Recording it because the first version of `lookahead_cost.js` swallowed those failures and divided by N, crediting each failure with zero cost and reporting a faster number than the truth.

`Infinite loop` is Showdown's own turn-loop guard. The likely cause is that a restored battle is not in `requestState === 'move'`, so `choose('default')` never advances it and the engine spins — which would make this a fault in how the fork is DRIVEN rather than in `fromJSON`, fixable by reading `requestState` and submitting real action strings. **That is a hypothesis, not a finding**, and testing it is the next thing to do.

Why this is a gate failure and not a detail: a search cannot evaluate a cell it cannot simulate. If a third of the matrix is unevaluable then the equilibrium is solved over a matrix with holes, and the value of a hole is not zero, it is unknown. The failure rate bounds the design as hard as the cost does, and it has to reach ~0 before G3 measures anything real.

1. **G3 — a real search recovers a meaningful share of G1.** Replace the oracle with an actual matrix-game evaluation and measure what fraction of +4.91 survives. Unmeasured.
2. **G4 — it wins games.** SPRT against the current greedy player, read as it goes. Unmeasured.

Next: the top-3 truncation rate, because it is cheap, it is a property of data we already hold, and it caps G3. `fit_joint.js` computes exactly this quantity for top-6 already; the same replay with a different K answers it, which is a parameter and not a second implementation.

Then G3. The honest expectation is that a real search recovers well under half of +4.91: the oracle knows the turn that happened, while a search must estimate it, average over a simultaneous opponent, and do both through a leaf that is itself only 63–64% accurate and dominated by `foe_alive`.

It does not claim the search will win. It claims one specific thing, measured: **there is +4.91 accuracy points of information one turn ahead, and there is +0.03 to +0.11 in the features we have been adding to the snapshot**. After a session in which three roadmap items dissolved under measurement and two of my own predictions were wrong, that is the only claim I am willing to put weight on — and the gates above exist so the next one has to earn the same way.

8. G2, resolved — 2026-08-03

Supersedes §6 G2 and the cost table in it. Three things were wrong and one of them was the reason the other two looked true.

8.1 The `Infinite loop` was not the driving, and not `fromJSON`

The hypothesis on record was that a restored battle is not in `requestState === 'move'`, so `choose('default')` never advances it. It is not that. The forked battle's `requestState` is "move" and the turn advances 4 -> 5 on the first fork. The driving was always fine.

`Infinite loop` is not a turn-loop guard. `sim/battle.ts:584` throws it when `log.length - sentLogPos > 1000` — a **log length** guard. And in the pinned checkout:

```
sim/state.ts:72 state.log = battle.log; // serialize ALIASES, it does not copy sim/state.ts:153 (battle as any).log
= state.log; // deserialize aliases straight back
```

`log` is the only key handled this way; every other array goes through `deserializeWithRefs`, which allocates fresh, and `hints` / `prng` are explicitly copied. So **every fork restored from one snapshot appended to one shared array**, log growth was cumulative across cells, and `sentLogPos` never moved because a forked battle has no stream to send to. At ~26 lines per turn that is ~38 forks before the guard fires — after which *every* remaining fork throws.

That is why the observed rates were 40/40, 27/40, 30/40, 24/40, 25/40: the failures were a **contiguous tail**, not scattered, and the rate tracked lines-per-turn rather than anything about the board. The tell was there in the original run — fork 33 threw, not fork 1 — and a rate alone cannot show it. `lookahead_cost.js` now reports whether failures are a tail or scattered, because the two have completely different causes.

The consequence that never threw is the worse one. `state.log = battle.log` aliases the *live* battle's log into the snapshot, so stepping a fork wrote simulated events into the log of the real game. A search that forks the battle it is playing would have corrupted the record of that game, silently.

Fixed in `engine/champions_sim.js` as `CS.snapshot()` / `CS.forkBattle()`, which detach the array at each boundary — what `fromJSON` already intends everywhere else. The pinned checkout is not edited (ADR-001). The 1000-line guard is deliberately left armed: with per-fork logs a fork accrues one turn of lines, so if it fires again it is a real runaway turn.

Result: 3,200 forks across 16 boards, 0 threw, 0 went nowhere, isolation clean on every board.

8.2 Two silent failure modes in the harness, both of which flattered the cost

Neither throws. Both were counted as successful cells.

- **Boards that never settle.** The drive loops are async and still writing choices when the turn counter reaches its target, so a snapshot could catch the battle between a choice and the turn it triggers. Restored from that moment there is no active request: `choose()` returns **true**, emits log lines, and does not advance. A search would score that cell as a successor when it is the same position. `measureBoard` now waits for `requestState === 'move'` with both sides holding a request, and skips the board otherwise. 9 of 25 boards are skipped — they ended under `default` play — so the measured boards are biased toward games that survive it, which is stated in the artifact.
- `turn > baseTurn` **is the wrong test for "it advanced."** Two outcomes advance the game without incrementing `turn`: `ended === true` (the turn ran and the battle finished) and `requestState === 'switch'` (something fainted and the engine wants a replacement). The first version of the check counted both as holes — discarding **terminal and post-KO positions specifically**, which are the highest-signal leaves a search has and the ones where the value is known exactly instead of estimated by a 63%-accurate k-NN.

8.3 The corrected cost, and the instrument's own noise floor

The "factor of nine, 3.5 to 32.2 ms" in §6 G2 **was the bug**: forks operating on an ever-growing shared array, plus boards that simulated nothing being credited as cheap cells.

5 back-to-back runs, 20 boards x 200 forks each, machine otherwise idle median fork cost 5.75 4.51 3.70 4.78 4.52 ms one run taken while the machine was busy 12.45 ms

So the cost is **~4.6 ms, spread ±22% run to run**, and ambient load can inflate it ~2.7x. Any verdict that turns on the difference between 4 ms and 12 ms is a verdict about machine load. Written to `data/lookahead-cost.json` and **read** by `truncation_curve.js` rather than retyped into it — the first draft of that file carried 9.68 as a literal and it was stale within the hour.

8.4 The truncation curve — `engine/truncation_curve.js` , the §6 "next thing"

One replay answers every K: `joint_rows.js`'s `pick` already sorts every candidate by the single-move score, so the chosen candidate's **rank** in that sort is free, and the rate at K is a suffix sum of the histogram. Same corpus, same ranker, same matcher. **K=6 reproduces 11.3% exactly**, which is the check that the instrument is the same one `fit_joint` used before any new number is read off it.

7,958 games, 86,715 joint turns, 173,430 slot decisions. Median 8 candidates per slot, max 11 — so §4.3's "144 joint actions per side" from an assumed 12 options overstates the unpruned matrix.

K joint miss per-slot miss cells cost @4.6ms verdict 3 52.2% 30.6% 81 0.37 s affordable 4 35.1% 19.4% 256 1.18 s viable idle, MARGINAL under load 5 21.5% 11.4% 625 2.88 s at budget in the best case 6 11.3% 5.9% 1296 5.96 s out

K=3 is safely affordable and misses the human's pair on 52.2% of turns. K=4 halves that to 35.1% for ~1.2 s and is the first real choice the budget allows — §6's "must prune to top-3" was decided on the inflated cost and is no longer forced.

8.5 What this does to G3, and the measurement that should come before it

The cap is real: at the affordable K the search never enumerates the human's pair on a third to a half of turns. Note the shape though — **per-slot miss is far lower than joint miss** (K=4: 19.4% vs 35.1%). The joint rate is high *because the matrix is the cross-product of two independent per-slot windows*, so a pair that is second-best on both slots is thrown away even when it is the best pair.

That is a property of how the pruning is built, not of the game. `magnemite.js:_decidePair` and the fitted joint vector already score **pairs**, including the interactions the per-slot ranker cannot see (Follow Me beside a setup move, spread damage beside a Protect). Taking the **top-M joint actions per side by the joint model** instead of the cross-product of two top-K windows costs the same cells for $M = K^2$ and should cover strictly more, because it is allowed to spend its budget unevenly.

That is the next measurement, and it is cheaper than G3 and caps it. It is the same replay again, scoring the joint candidates with the joint vector and asking where the human's pair ranks. It also reframes the undecided feature H2H a second time: §4.3 argued a better prior is worth more as a pruner than as a player, and this is the specific quantity that would show it.